gRPC: a Go server and a Dart client

One unary call, one stream, a deadline, and a cancel

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Week 12

TODAY

What this lab is for

Call a Go server once

Generate a Dart client from a .proto file, call a unary RPC with a deadline, and read back a DeadlineExceeded status when the server is slow.

Watch a stream live

Subscribe to a server-streaming RPC, show one reading a second on screen, and cancel it cleanly with a button.

TIME

2 hours

SUBMIT

Push to your team repo, branch `lab-6`, before next lab

Install the toolchain

Install these once per machine, then branch the team repository for this lab.

```
brew install protobuf   # or: apt install protobuf-compiler

go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

dart pub global activate protoc_plugin
```

Add them to PATH

The Go plugins land in `GOPATH/bin`, the Dart one in your pub-cache `bin` folder. Neither is on PATH by default.

SETUP

Where your code goes

1. Branch the team repository:

```
git checkout -b lab-6
```

2. Put the contract in `proto/readings/v1/`.
3. Server code in `server/lab6/main.go`. App code in `lib/lab6/`.
4. Run the server once before writing any Dart code.

Order of work

Contract, then server, then client. Do not write a widget before `go run` prints a listening line in your terminal.

SETUP

The contract: readings.proto

```
syntax = "proto3";
package readings.v1;
message Reading {
  string device_id    = 1;
  double value         = 2;
  int64  taken_at_unix = 3;
}
message DeviceRequest { string device_id = 1; }
service Readings {
  rpc GetLatest(DeviceRequest) returns (Reading);
  rpc StreamReadings(DeviceRequest) returns (stream Reading);
}
```

GetLatest returns one Reading. StreamReadings returns a stream of them.

TASK I

Generate the stubs, run the server

1. Run protoc for Go, then for Dart (the exact commands are next).
2. Add the packages: `flutter pub add grpc protobuf`.
3. Start the server: `go run ./server/lab6`.
4. Confirm it prints the port it is listening on.

DONE WHEN

- [ ] `server/lab6/gen/` holds two generated Go files.
- [ ] `lib/gen/` holds two generated Dart files.
- [ ] The server keeps running with no error printed.
- [ ] `flutter pub get` resolves cleanly.

Generated code is build output. Never hand-edit it. Change the .proto file and regenerate instead.

TASK I

Generating stubs for Go and Dart

Run protoc once per language, against the same .proto file.

```
protoc -Iproto \
  --go_out=gen      --go_opt=paths=source_relative \
  --go-grpc_out=gen --go-grpc_opt=paths=source_relative \
  proto/readings/v1/readings.proto

protoc -Iproto --dart_out=grpc:lib/gen \
  proto/readings/v1/readings.proto
```

Commit the generated files, or run protoc in CI. Either way, pick one and write it down.

TASK I

A Go server: struct and GetLatest

```
type server struct {
    pb.UnimplementedReadingsServer
    latest *pb.Reading
}

func (s *server) GetLatest(
    ctx context.Context, req *pb.DeviceRequest,
) (*pb.Reading, error) {
    if s.latest == nil {
        return nil,
     status.Error(codes.NotFound, "empty")
    }
    return s.latest, nil
}
```

Embed the Unimplemented type. It gives every method a stub, so the server compiles before GetLatest exists.

No reading yet is an error, not a zero value: NotFound tells the client empty and absent apart.

TASK I

A Go server: the stream and main

```
func (s *server) StreamReadings(
    req *pb.DeviceRequest,
    stream pb.Readings_StreamReadingsServer,
) error {
    ctx := stream.Context()
    for ctx.Err() == nil {
        time.Sleep(time.Second)
        stream.Send(s.latest)
    }
    return nil
}
```

```
func main() {
    lis, _ := net.Listen(
        "tcp", ":50051")
    gs := grpc.NewServer()

    pb.RegisterReadingsServer(
        gs, &server{})
    gs.Serve(lis)
}
```

StreamReadings loops until the client's context is done. main wires the implementation to a TCP listener.

TASK II

A unary call with a deadline

1. Add a debug flag that sleeps 3 seconds before `GetLatest` returns.
2. Call `getLatest` from Flutter with a 2-second `CallOptions` timeout.
3. Catch `GrpcError` and show its status code on screen.
4. Confirm the call fails while the server is still asleep.

DONE WHEN

- [ ] The screen shows an error, not a stuck spinner.
- [ ] The error code is `deadlineExceeded`.
- [ ] Turning off the sleep makes the same call succeed.

A deadline is not optional. Without one, a slow server holds your UI state forever.

A Dart client: the unary call

```
final channel = ClientChannel(
    '10.0.2.2', port: 50051,
    options: const ChannelOptions(
        credentials:
    ChannelCredentials.insecure()));
final client = ReadingsClient(channel); try {
  final r = await client.getLatest(
    DeviceRequest()..deviceId = id,
    options: CallOptions(
        timeout: const Duration(seconds: 2)));
  setState(() => _value = r.value);
} on GrpcError catch (e) {
  setState(() => _error = e.code); }
```

Insecure means plaintext, fine for a lab server on your own machine, never for a real deployment.

10.0.2.2 is the Android emulator's address for your host machine, not `localhost`.

TASK III

The streaming screen, with a cancel

1. Call `streamReadings` with the device id.
2. Listen on the stream and update the screen on each `Reading`.
3. Add a Cancel button that stops the subscription.
4. Show a stopped label once canceled.

DONE WHEN

- [ ] The value on screen changes about once a second.
- [ ] Cancel stops the updates immediately.
- [ ] Nothing is logged as an error after a clean cancel.

TASK III

A Dart client: the stream and cancel

```
final call = client.streamReadings(
  DeviceRequest()..deviceId = id,
);
final sub = call.listen(
  (r) => setState(() => _value = r.value),
  onError: (_) => setState(() => _live =
    false),
);

void _cancel() {
  call.cancel();   // stop the stream
  sub.cancel();
  setState(() => _live = false);
}
```

call is a ResponseStream. Canceling it tells the server to stop sending, not only to stop listening locally.

The timeout still applies. Pick a generous one for a stream you plan to keep open for a while.

ACCEPTANCE

What the grader will do

THE DEMO

- [ ] The stream updates live, about once a second.
- [ ] Pressing Cancel stops it immediately.
- [ ] A 2-second deadline fails `getLatest` against a server sleeping 3 seconds, with `deadlineExceeded`.

IN THE REPOSITORY

The .proto file, the server, the app code, and a README naming your device id scheme and the port you used.

One commit per task, on branch `lab-6`.

TROUBLESHOOTING

Errors you will meet, and their fixes

`protoc-gen-go: program not found`	Not on PATH. Add both plugin bin folders to PATH.
`Target of URI doesn't exist: readings.pb.dart`	Match `--dart_out` to where the app imports it.
`Connection refused to localhost:50051`	Emulator localhost is itself. Use `10.0.2.2`.
`UNAVAILABLE: failed to connect`	The server isn't running, or the port is wrong.
`CLEARTEXT communication not permitted`	Android blocks it. Set `usesCleartextTraffic`.

Push before you leave.

Branch lab-6 · one commit per task · README with device id scheme and port